Parâmetros, Retorno e Escopo

Funções com parâmetros, return, valores padrão e escopo

Aula 16 • 27/10/2026 • Linguagem de Programação Python (80h) • CETAM

Profa. Priscila Gonçalves

Visão geral da aula

Data

Terça-feira, 27/10/2026
Aula 16 • 4 tempos
Laboratório de informática

Habilidade técnica

Aplicar o recurso de modularização e métodos para o uso de funções e parâmetros.

Projeto

buscar_destino(nome) e validar_orcamento(total, limite) no menu

Habilidade técnica

- Aplicar o recurso de modularização e métodos para o uso de funções e parâmetros
- Passar parâmetros e receber valores com return
- Usar valores padrão e entender o escopo das variáveis
- Integrar novas funções ao menu do sistema

Objetos de conhecimento

- Funções em Python: criação e chamada de funções
- Passagem de parâmetros e retorno
- Definindo funções com def()
- Parâmetros, argumentos e retorno

Roteiro dos 4 tempos

1º tempo

Aula expositiva: passagem de parâmetros e retorno

2º tempo

Modularização e métodos para o uso de funções

3º tempo

Parâmetros x argumentos, return x print, valores padrão, escopo e atividade

4º tempo

Desafio: buscar_destino() e validar_orcamento() no menu

1

1º tempo • Parâmetros e retorno

Aula expositiva e dialogada

Retomada da Aula 15

- def cria a função; o nome a chama
- Parâmetros levam dados para dentro da função
- return devolve um resultado para quem chamou
- Modularizar evita repetição e organiza o código
- Hoje: aprofundar parâmetros, retorno e escopo

Passagem de parâmetros

- Parâmetros levam dados para dentro da função
- Parâmetro: nome na definição (def f(nome))
- Argumento: valor na chamada (f("Ana"))
- Os argumentos são associados aos parâmetros
- Podem ser valores, variáveis ou expressões

Argumentos posicionais

```
def apresentar(nome, destino):
    print(nome, "vai a", destino)

apresentar("Ana", "Tefé")
apresentar("Tefé", "Ana")   # trocado
```

- Associação pela posição
- Primeiro argumento: nome
- A ordem trocada muda o sentido

Argumentos nomeados

```
apresentar(destino="Tefé", nome="Ana")
apresentar(nome="Caio", destino="Coari")
```

- Nome do parâmetro = valor
- A ordem deixa de importar
- Mais legível em funções longas

Retorno de um valor

```
def calcular_total(dias, gasto):
    return dias * gasto

total = calcular_total(4, 150)
print(total)                    # 600
print(calcular_total(2, 80))    # 160
```

- return devolve o resultado
- Guarde em uma variável
- A função termina no return

Retorno de vários valores

```
def resumo(gastos):
    return min(gastos), max(gastos)

menor, maior = resumo([120, 80, 200])
print(menor, maior)   # 80 200
```

- Vírgula devolve uma tupla
- Desempacote em variáveis
- Útil para vários resultados

Funções sem retorno

```
def mostrar(msg):
    print(msg)

r = mostrar("Olá")
print(r)   # None
```

- Sem return, devolve None
- Serve para mostrar na tela
- Para calcular, use return

Caminho de uma chamada com retorno



Argumentos entram pela chamada; o resultado volta pelo return.

Erros comuns

- TypeError: faltam ou sobram argumentos na chamada
- Argumentos na ordem errada
- Usar print quando precisava de return
- Esquecer de guardar o valor retornado
- Usar um parâmetro fora da função

2

2º tempo • Modularização

Aplicar o recurso de modularização e métodos

Modularização: dividir para conquistar

- Cada função resolve um problema pequeno
- O programa principal só organiza as chamadas
- Cada função pode ser testada isoladamente
- Funções podem ser reaproveitadas em outros programas
- Na Aula 15: calcular_total, exibir_menu e cadastrar_cliente

Antes: tudo junto

```
dias = int(input("Dias: "))
gasto = float(input("Gasto/dia: "))
total = dias * gasto
if total <= 1000:
    print("Dentro do orçamento")
else:
    print("Fora do orçamento")
```

- Funciona, mas mistura tudo
- Difícil de reaproveitar
- Difícil de testar por partes

Depois: com funções

```
def ler_dados():
    dias = int(input("Dias: "))
    gasto = float(input("Gasto/dia: "))
    return dias, gasto

def validar_orcamento(total, limite):
    return total <= limite
```

- Cada função, uma tarefa
- ler_dados devolve dois valores
- validar devolve True ou False

Funções chamando funções

```
def calcular_total(dias, gasto):
    return dias * gasto

def relatorio(dias, gasto):
    total = calcular_total(dias, gasto)
    return f"Total: R$ {total:.2f}"

print(relatorio(4, 150))
```

- Uma função usa outra
- Saída: Total: R$ 600.00
- Reaproveitamento na prática

Boas práticas ao criar funções

- Nome de verbo que explique a tarefa
- Uma função, uma responsabilidade
- Receba dados por parâmetros e devolva com return
- Evite print dentro de funções de cálculo
- Comente o que a função faz e o que devolve

3

3º tempo • Conceitos e prática

Parâmetros, return, valores padrão e escopo

Parâmetros × argumentos

Parâmetro

Variável na definição da função.
def saudar(nome):

Argumento

Valor passado na chamada.
saudar("Ana")

return × print

print

Mostra o valor na tela.
O valor não pode ser reutilizado.

return

Devolve o valor para quem chamou.
Pode ser guardado e calculado.

return x print na prática

```
def soma_print(a, b):
    print(a + b)

def soma_return(a, b):
    return a + b

x = soma_return(2, 3) * 10  # 50
y = soma_print(2, 3)        # None
```

- soma_print mostra 5
- soma_return permite continuar
- y recebe None

Valores padrão

```
def desconto(preco, taxa=0.1):
    return preco * (1 - taxa)

print(desconto(200))        # 180.0
print(desconto(200, 0.25))   # 150.0
```

- taxa tem valor padrão
- O argumento é opcional
- Padrões ficam depois dos obrigatórios

Escopo local

```
def mostrar():
    total = 5      # local
    print(total)

total = 100       # global
mostrar()          # 5
print(total)       # 100
```

- Variável criada na função é local
- Só existe dentro dela
- Não altera a global de mesmo nome

Local não existe fora

```
def calcular():
    resultado = 10

calcular()
print(resultado)   # NameError
```

- resultado só existe dentro
- Fora da função: NameError
- Para usar fora, devolva com return

Escopo global e global

```
contador = 0

def somar():
    global contador
    contador += 1

somar()
print(contador)   # 1
```

- global altera a variável externa
- Sem global: UnboundLocalError
- Prefira parâmetros e return

Atividade avaliativa prática

- Funções com retorno de valores e de listas
- 1. Função que devolve um valor (média de notas)
- 2. Função que devolve uma lista (gastos acima do limite)
- Entrega: arquivo .py com chamadas de teste
- Use parâmetros, return e nomes claros

Exercício 1: retorno de valor

```
def media(notas):
    return sum(notas) / len(notas)

print(media([8, 7, 9]))   # 8.0
```

- Recebe uma lista
- Devolve um número
- Teste com outras notas

Exercício 2: retorno de lista

```
def gastos_acima(gastos, limite):
    acima = []
    for g in gastos:
        if g > limite:
            acima.append(g)
    return acima

print(gastos_acima([50, 200, 90, 300], 100))
```

- Devolve uma nova lista
- Saída: [200, 300]
- Teste com limites diferentes

Critérios da atividade

- Funções definidas com def e chamadas corretamente
- Parâmetros usados no lugar de valores fixos
- return em vez de print nas funções de cálculo
- Valor e lista devolvidos corretamente
- Nomes claros e testes com valores diferentes

4

4º tempo • Desafio do projeto

buscar_destino() e validar_orcamento() no menu

Desafio: funções integradas ao menu

- buscar_destino(nome): devolve o preço do destino, ou None
- validar_orcamento(total, limite): devolve True ou False
- Integrar as duas funções à opção do menu
- Mostrar se o destino está dentro ou fora do orçamento
- Dados do catálogo são um exemplo

buscar_destino(nome)

```
DESTINOS = {"Parintins": 850.0, "Tefé": 600.0}

def buscar_destino(nome):
    nome = nome.strip().title()
    if nome in DESTINOS:
        return DESTINOS[nome]
    return None
```

validar_orcamento(total, limite)

```
def validar_orcamento(total, limite):
    return total <= limite

print(validar_orcamento(850, 1000))   # True
print(validar_orcamento(1200, 1000))  # False
```

Integração com o menu

```
while True:
    exibir_menu()
    op = input("Escolha: ")
    if op == "0":
        break
    elif op == "1":
        preco = buscar_destino(input("Destino: "))
        if preco is None:
            print("Destino não encontrado")
        elif validar_orcamento(preco, 1000):
            print("Dentro do orçamento")
        else:
            print("Fora do orçamento")
```

Dicas para o desafio

- Teste cada função sozinha antes de ligar ao menu
- Use return nas funções, e print só no menu
- Trate o caso de destino que não existe (None)
- Use o limite como parâmetro, e não valor fixo
- Reaproveite exibir_menu() da Aula 15

Fechamento

- Hoje: parâmetros, argumentos, return e escopo
- Atividade: funções que devolvem valor e lista
- Desafio: buscar_destino() e validar_orcamento() no menu
- Dúvidas?
- Obrigada!